

```
In [1]: # This Python 3 environment comes with many helpful analytics libraries installed
# It is defined by the kaggle/python Docker image: https://github.com/kaggle/docker-python
# For example, here's several helpful packages to load

import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

# Input data files are available in the read-only "../input/" directory
# For example, running this (by clicking run or pressing Shift+Enter) will list all files under the input directory

import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))

# You can write up to 20GB to the current directory (/kaggle/working/) that gets preserved as output when you create a version using "Save & Run All"
# You can also write temporary files to /kaggle/temp/, but they won't be saved outside of the current session

# Use the kagglehub client library to attach Kaggle resources like competitions, datasets, and models to your session
# Learn more about kagglehub: https://github.com/Kaggle/kagglehub/blob/main/README.md

import kagglehub
# kagglehub.dataset_download('<owner>/<dataset-slug>')
```

/kaggle/input/datasets/vishweshhampali/mavendb/maven.duckdb

```
In [2]: !pip install duckdb kaleido==0.2.1 --quiet

import duckdb
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots
import plotly.io as pio

# Set renderer to static PNG – makes charts embed in notebook
pio.renderers.default = 'png'

DB_PATH = "/kaggle/input/datasets/vishweshhampali/mavendb/maven.duckdb"
con = duckdb.connect(DB_PATH, read_only=True)

print("Available tables:")
print(con.execute("SHOW TABLES").df())
```

Available tables:

	name
0	mart_channel_performance
1	mart_channel_roi
2	mart_cohort_retention
3	mart_conversion_funnel
4	mart_crosssell
5	mart_funnel_trend
6	mart_page_leak
7	mart_refund_rates
8	mart_revenue_trend
9	mart_user_intent
10	mart_user_segments
11	order_item_refunds
12	order_items
13	orders
14	products
15	stg_order_items
16	stg_orders
17	stg_pageviews
18	stg_products
19	stg_refunds
20	stg_sessions
21	website_pageviews
22	website_sessions

```
In [3]: kpi_sql = """
SELECT
    SUM(sessions)                AS total_sessions,
    SUM(orders)                  AS total_orders,
    ROUND(SUM(revenue), 2)       AS total_revenue,
    ROUND(SUM(orders) * 100.0 / NULLIF(SUM(sessions), 0), 2) AS overall_cvr_pct
FROM mart_revenue_trend
"""

kpi = con.execute(kpi_sql).df()
print(kpi.to_string(index=False))
```

total_sessions	total_orders	total_revenue	overall_cvr_pct
472871.0	32313.0	1938509.75	6.83

```
In [4]: trend_sql = """
SELECT
    month,
    SUM(revenue) AS monthly_revenue
FROM mart_revenue_trend
GROUP BY month
ORDER BY month
"""

trend = con.execute(trend_sql).df()
trend['month'] = pd.to_datetime(trend['month'])

fig = px.line(
    trend,
    x='month',
    y='monthly_revenue',
    title='Monthly Revenue Trend – Maven Fuzzy Factory (2012-2015)',
```

```

labels={'month': 'Month', 'monthly_revenue': 'Revenue ($)'},
template='plotly_white'
)

fig.update_traces(line=dict(color='#1a1a1a', width=2.5))
peak_x = pd.Timestamp('2014-12-01').timestamp() * 1000
fig.add_vline(
    x=peak_x,
    line_dash='dash',
    line_color='#dc2626',
    annotation_text='Peak Dec 2014',
    annotation_position='top left',
    annotation_font_color='#dc2626'
)
fig.update_layout(
    font_family='monospace',
    title_font_size=14,
    hovermode='x unified'
)
fig.show()

```

Monthly Revenue Trend – Maven Fuzzy Factory (2012–2015)



```

In [5]: channel_sql = """
SELECT
    utm_source,
    device_type,
    SUM(sessions)                AS sessions,
    ROUND(AVG(cvr_pct), 2)      AS cvr_pct,
    ROUND(AVG(revenue_per_session), 2) AS rev_per_session
FROM mart_channel_roi
GROUP BY utm_source, device_type

```

```
ORDER BY rev_per_session DESC
"""
```

```
channel = con.execute(channel_sql).df()
channel['utm_source'] = channel['utm_source'].fillna('direct')
print(channel.to_string(index=False))
```

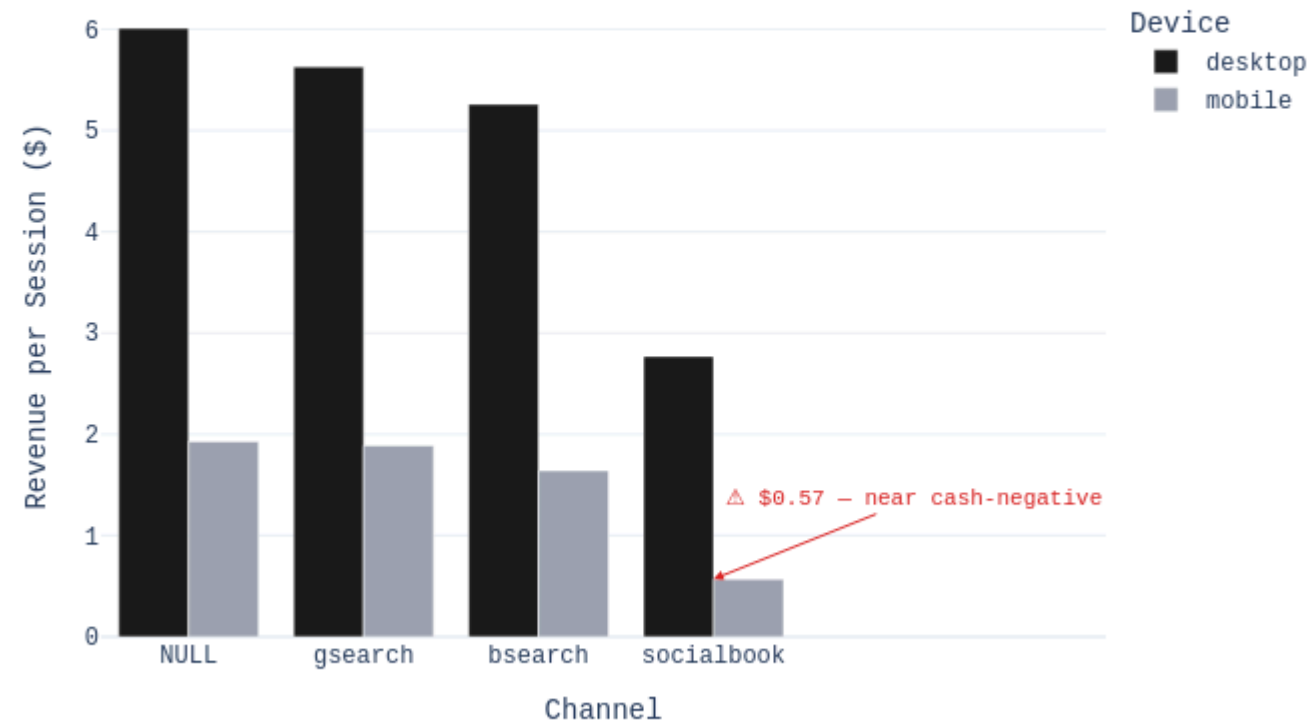
utm_source	device_type	sessions	cvr_pct	rev_per_session
NULL	desktop	51576.0	9.89	6.01
gsearch	desktop	215077.0	9.38	5.63
bsearch	desktop	54262.0	8.71	5.26
socialbook	desktop	6112.0	4.21	2.77
NULL	mobile	31752.0	3.20	1.93
gsearch	mobile	100958.0	3.13	1.89
bsearch	mobile	8561.0	2.70	1.64
socialbook	mobile	4573.0	0.83	0.57

```
In [6]: fig = px.bar(
    channel,
    x='utm_source',
    y='rev_per_session',
    color='device_type',
    barmode='group',
    title='Revenue per Session by Channel & Device Type',
    labels={
        'utm_source': 'Channel',
        'rev_per_session': 'Revenue per Session ($)',
        'device_type': 'Device'
    },
    color_discrete_map={'desktop': '#1a1a1a', 'mobile': '#9ca3af'},
    template='plotly_white'
)

# Annotate the problem bar
fig.add_annotation(
    x='socialbook', y=0.57,
    text='⚠ $0.57 – near cash-negative',
    showarrow=True,
    arrowhead=2,
    arrowcolor='#dc2626',
    font=dict(color='#dc2626', size=11),
    ax=100, ay=-40
)

fig.update_layout(
    font_family='monospace',
    title_font_size=14,
    legend_title_text='Device'
)
fig.show()
```

Revenue per Session by Channel & Device Type

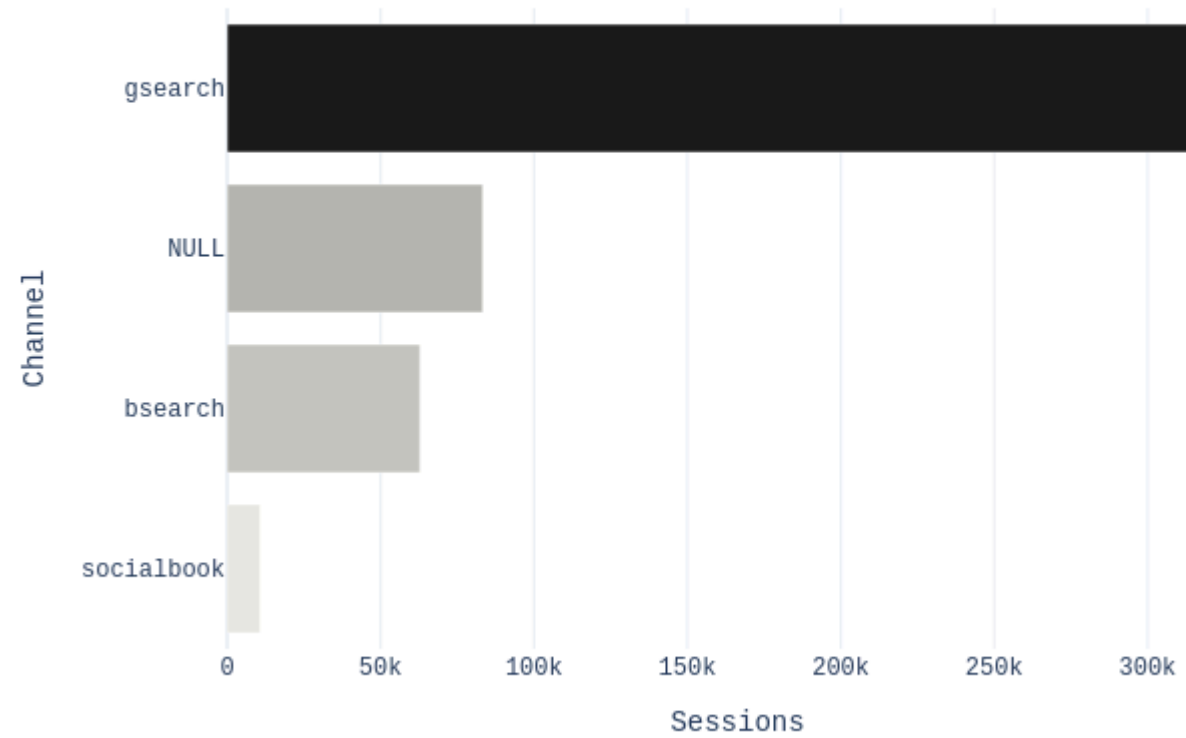


```
In [7]: sessions_sql = """
SELECT
    COALESCE(utm_source, 'direct') AS channel,
    SUM(sessions) AS total_sessions
FROM mart_channel_roi
GROUP BY utm_source
ORDER BY total_sessions DESC
"""

sessions = con.execute(sessions_sql).df()

fig = px.bar(
    sessions,
    x='total_sessions',
    y='channel',
    orientation='h',
    title='Total Sessions by Channel',
    labels={'total_sessions': 'Sessions', 'channel': 'Channel'},
    color='total_sessions',
    color_continuous_scale=[ '#e8e8e2', '#1a1a1a' ],
    template='plotly_white'
)
fig.update_layout(
    font_family='monospace',
    title_font_size=14,
    coloraxis_showscale=False,
    yaxis={'categoryorder': 'total ascending'}
)
fig.show()
```

Total Sessions by Channel



```
In [8]: funnel_sql = """
SELECT
    SUM(sessions)          AS sessions,
    SUM(reached_product)  AS reached_product,
    SUM(reached_cart)     AS reached_cart,
    SUM(reached_shipping) AS reached_shipping,
    SUM(reached_billing)  AS reached_billing,
    SUM(converted)        AS converted
FROM mart_funnel_trend
"""

funnel_raw = con.execute(funnel_sql).df()

# Reshape to Long format for funnel chart
stages = ['sessions', 'reached_product', 'reached_cart',
          'reached_shipping', 'reached_billing', 'converted']
labels = ['Sessions', 'Reached Product', 'Reached Cart',
          'Reached Shipping', 'Reached Billing', 'Converted']
values = [int(funnel_raw[s].iloc[0]) for s in stages]

fig = go.Figure(go.Funnel(
    y=labels,
    x=values,
    textposition='inside',
    textinfo='value+percent initial',
    marker=dict(color=[
        '#1a1a1a', '#dc2626', '#6b7280',
        '#9ca3af', '#b8bec6', '#16a34a'
    ]),
)),
```



```

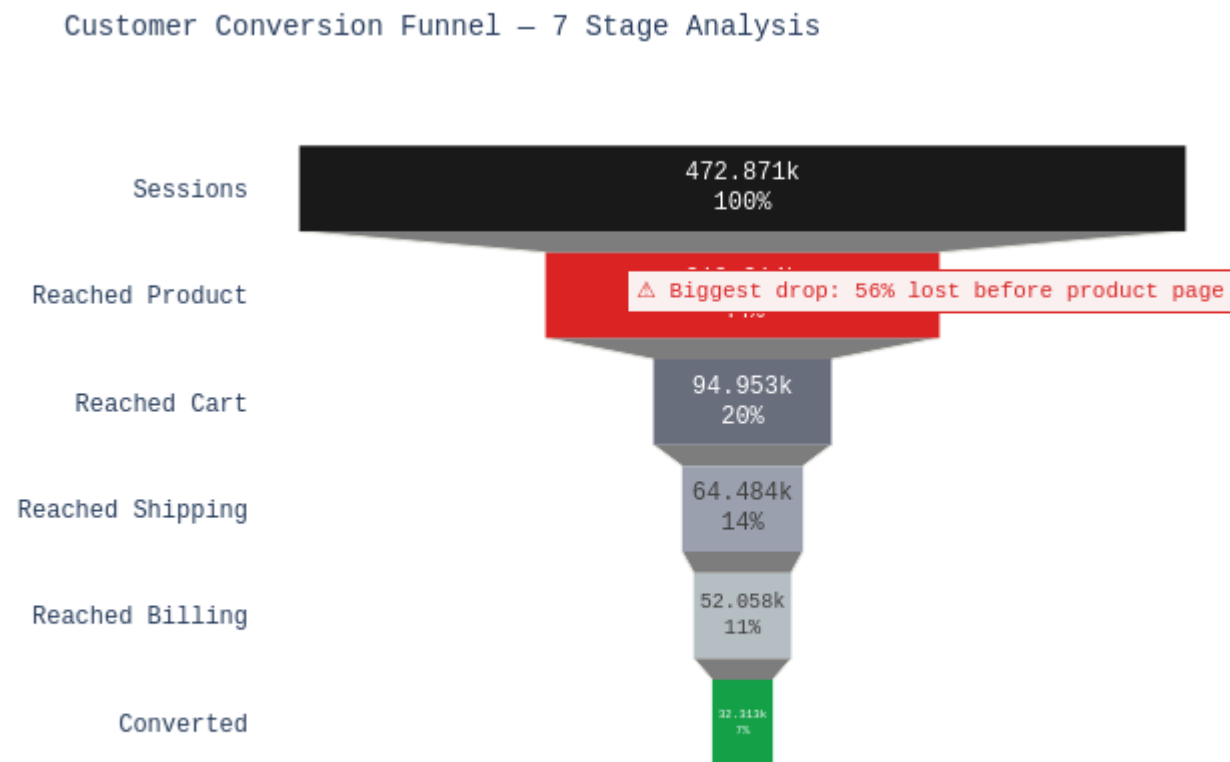
connector=dict(line=dict(color='#e8e8e2', width=1))
))

fig.update_layout(
    title='Customer Conversion Funnel – 7 Stage Analysis',
    font_family='monospace',
    title_font_size=14,
    template='plotly_white'
)

fig.add_annotation(
    text='△ Biggest drop: 56% lost before product page',
    x=1, y=0.79,
    xref='paper', yref='paper',
    showarrow=False,
    font=dict(color='#dc2626', size=11),
    bgcolor='#fef2f2',
    bordercolor='#dc2626',
    borderwidth=1,
    borderpad=4
)

fig.show()

```



```

In [9]: leak_sql = """
SELECT
    COALESCE(utm_source, 'direct') AS channel,
    ROUND(SUM(revenue_if_10pct_recovered), 0) AS recoverable_rev
FROM mart_page_leak
GROUP BY utm_source

```

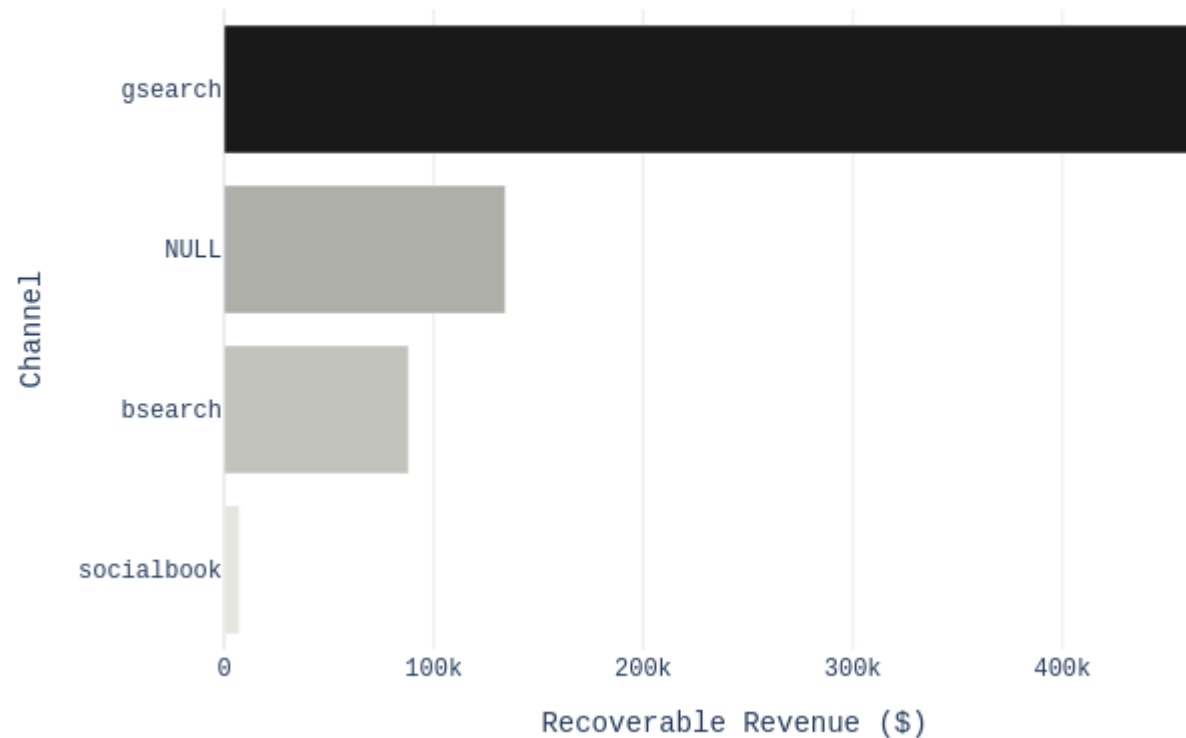
```
ORDER BY recoverable_rev DESC
"""

leak = con.execute(leak_sql).df()
total = leak['recoverable_rev'].sum()
print(f"Total recoverable revenue at 10% fix: ${total:,.0f}")

fig = px.bar(
    leak,
    x='recoverable_rev',
    y='channel',
    orientation='h',
    title=f'Recoverable Revenue at 10% Product Page Fix – Total: ${total:,.0f}',
    labels={'recoverable_rev': 'Recoverable Revenue ($)', 'channel': 'Channel'},
    color='recoverable_rev',
    color_continuous_scale=['#e8e8e2', '#1a1a1a'],
    template='plotly_white'
)
fig.update_layout(
    font_family='monospace',
    title_font_size=14,
    coloraxis_showscale=False,
    yaxis={'categoryorder': 'total ascending'}
)
fig.show()
```

Total recoverable revenue at 10% fix: \$692,066

Recoverable Revenue at 10% Product Page Fix – Total: \$692,066



In [10]: cohort_sql = """
SELECT


```

        cohort_month,
        months_since_first,
        AVG(active_users) AS avg_active_users
FROM mart_cohort_retention
GROUP BY cohort_month, months_since_first
ORDER BY cohort_month, months_since_first
"""

cohort = con.execute(cohort_sql).df()
cohort['cohort_month'] = pd.to_datetime(cohort['cohort_month'])
cohort['cohort_label'] = cohort['cohort_month'].dt.strftime('%b %Y')

# Normalise to % of month-0 users
cohort['month_0'] = cohort.groupby('cohort_label')['avg_active_users'].transform('first')
cohort['retention_pct'] = (cohort['avg_active_users'] / cohort['month_0'] * 100).round(1)

fig = px.line(
    cohort,
    x='months_since_first',
    y='retention_pct',
    color='cohort_label',
    title='Cohort Retention Curves – % of Users Returning by Month',
    labels={
        'months_since_first': 'Months Since First Visit',
        'retention_pct': 'Retention (%)',
        'cohort_label': 'Cohort'
    },
    template='plotly_white'
)

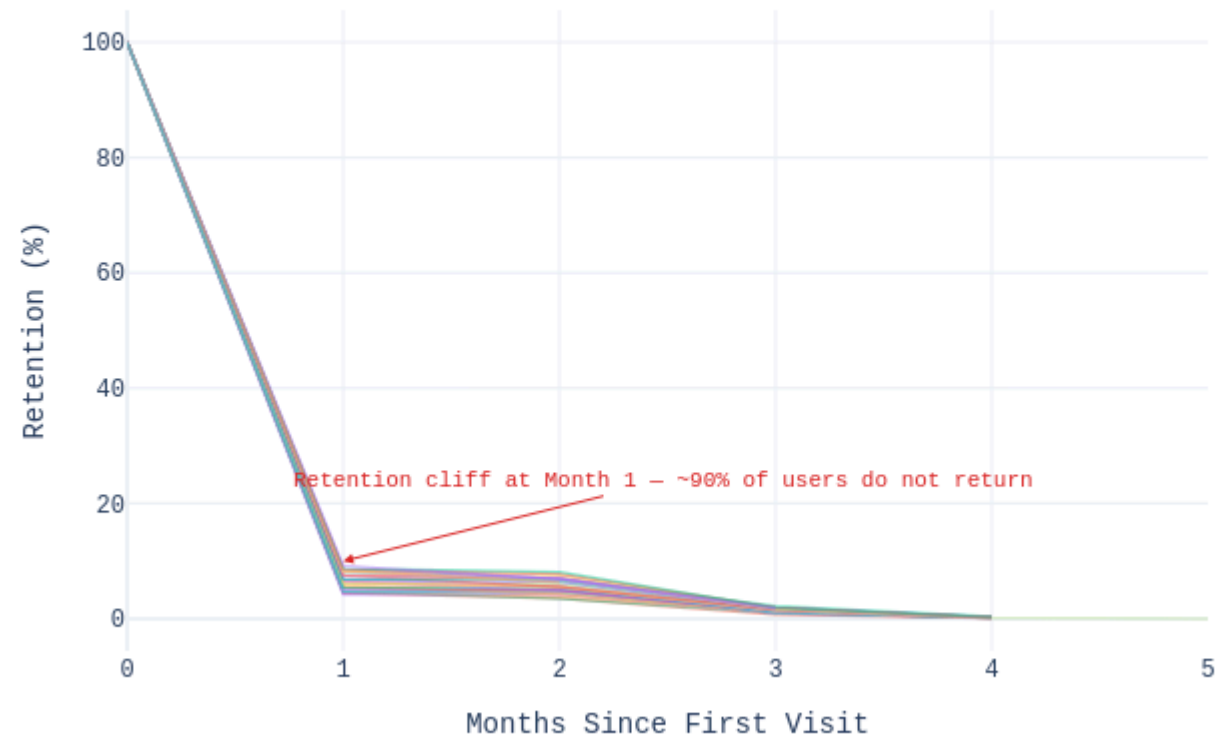
fig.update_traces(opacity=0.5, line=dict(width=1.2))
fig.update_layout(
    font_family='monospace',
    title_font_size=14,
    showlegend=False, # too many cohorts – hides cleanly
    hovermode='x unified'
)

fig.add_annotation(
    text='Retention cliff at Month 1 – ~90% of users do not return',
    x=1, y=10,
    showarrow=True,
    arrowhead=2,
    arrowcolor='#dc2626',
    font=dict(color='#dc2626', size=11),
    ax=160, ay=-40
)

fig.show()

```

Cohort Retention Curves – % of Users Returning by Month



```
In [11]: m1_sql = """
SELECT
    cohort_month,
    AVG(active_users) AS returning_users
FROM mart_cohort_retention
WHERE months_since_first = 1
GROUP BY cohort_month
ORDER BY cohort_month
"""

m1 = con.execute(m1_sql).df()
m1['cohort_month'] = pd.to_datetime(m1['cohort_month'])

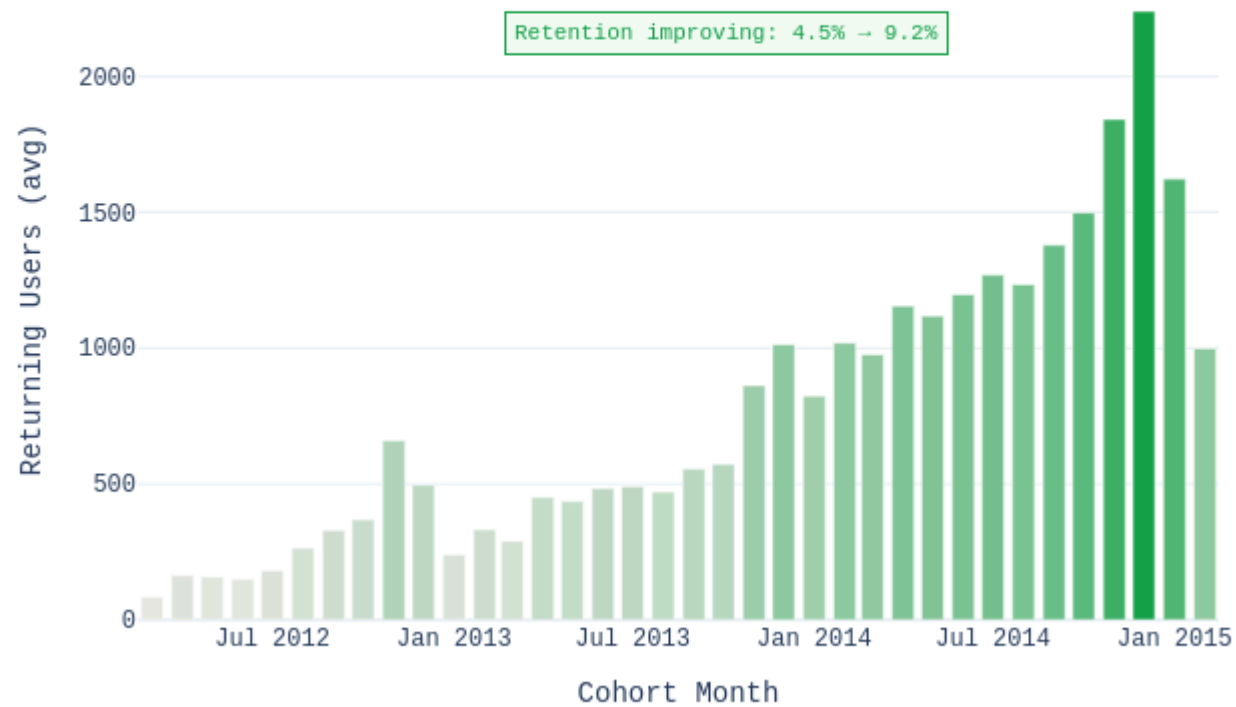
fig = px.bar(
    m1,
    x='cohort_month',
    y='returning_users',
    title='Month-1 Return Rate by Cohort – Improving Over Time',
    labels={'cohort_month': 'Cohort Month', 'returning_users': 'Returning Users (avg)'},
    color='returning_users',
    color_continuous_scale=['#e8e8e2', '#16a34a'],
    template='plotly_white'
)
fig.update_layout(
    font_family='monospace',
    title_font_size=14,
    coloraxis_showscale=False
)
fig.add_annotation(
```

```

text='Retention improving: 4.5% → 9.2%',
x=0.75, y=0.95,
xref='paper', yref='paper',
showarrow=False,
font=dict(color='#16a34a', size=11),
bgcolor='#f0fdf4',
bordercolor='#16a34a',
borderwidth=1,
borderpad=4
)
fig.show()

```

Month-1 Return Rate by Cohort – Improving Over Time



```

In [12]: # =====
# CELL 11 – Product: Refund Rates & Cross-sell
# =====

product_sql = """
SELECT
    r.product_name,
    ROUND(AVG(r.refund_rate_pct), 1) AS refund_rate_pct,
    ROUND(AVG(c.crosssell_rate_pct), 1) AS crosssell_rate_pct
FROM    mart_refund_rates r
LEFT JOIN mart_crosssell    c
        ON r.product_name = c.entry_product
GROUP BY r.product_name
ORDER BY crosssell_rate_pct DESC
"""

product = con.execute(product_sql).df()

```

```

fig = make_subplots(
    rows=2, cols=1,
    subplot_titles=(
        'Refund Rate by Product (%)',
        'Cross-sell Rate by Product (%)'
    ),
    vertical_spacing=0.18
)

colors_refund = ['#dc2626' if r >= 5 else '#d97706' if r >= 3 else '#16a34a'
                  for r in product['refund_rate_pct']]

fig.add_trace(go.Bar(
    x=product['refund_rate_pct'],
    y=product['product_name'],
    orientation='h',
    marker_color=colors_refund,
    name='Refund Rate',
    text=product['refund_rate_pct'].astype(str) + '%',
    textposition='outside'
), row=1, col=1)

colors_cross = ['#16a34a' if c >= 80 else '#374151'
                 for c in product['crosssell_rate_pct']]

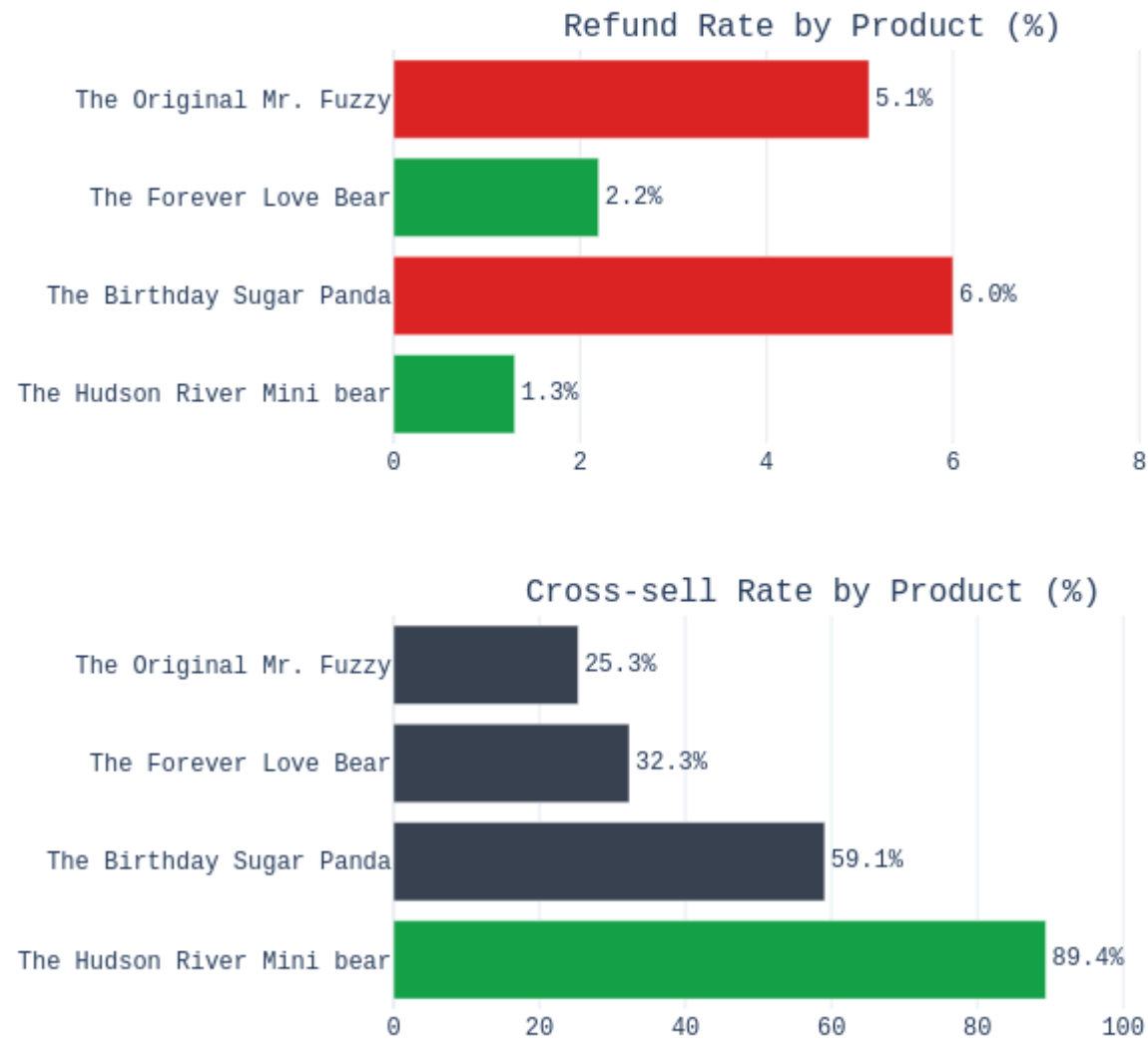
fig.add_trace(go.Bar(
    x=product['crosssell_rate_pct'],
    y=product['product_name'],
    orientation='h',
    marker_color=colors_cross,
    name='Cross-sell Rate',
    text=product['crosssell_rate_pct'].astype(str) + '%',
    textposition='outside'
), row=2, col=1)

fig.update_xaxes(range=[0, 9], row=1)
fig.update_xaxes(range=[0, 115], row=2)

fig.update_layout(
    title='Product Health – Refund Rate vs Cross-sell Rate',
    font_family='monospace',
    title_font_size=14,
    showlegend=False,
    template='plotly_white',
    height=600,
    margin=dict(l=180, r=80, t=80, b=40)
)
fig.show()

```

Product Health – Refund Rate vs Cross-sell Rate



```
In [13]: recs = pd.DataFrame({
  'Priority': ['P1', 'P1', 'P2', 'P2', 'P3'],
  'Action': [
    'Fix product page UX',
    'Cut Socialbook mobile spend',
    'Add Mini Bear as checkout upsell',
    'Investigate Birthday Sugar Panda',
    'Build email retention programme'
  ],
  'Key Metric': [
    '56% drop-off at product page',
    '$0.57/session vs $5.73 best channel',
    '89% cross-sell rate',
    '6% refund rate – highest in range',
    '<10% month-1 return rate'
  ],
  'Expected Impact': [
    '~$691K recoverable at 10% fix',
    '10x revenue gap vs best channel',
    'Zero-cost revenue lift',
    'Reduce returns, protect margin',
    '↑ LTV per acquired customer'
  ]
})
```

```
})

print("\n=== RECOMMENDATIONS ===\n")
print(recs.to_string(index=False))
```

=== RECOMMENDATIONS ===

Priority	Action	Key Metric	Expected Impact
P1	Fix product page UX	56% drop-off at product page	~\$691K recoverable at 10% fix
P1	Cut Socialbook mobile spend	\$0.57/session vs \$5.73 best channel	10x revenue gap vs best channel
P2	Add Mini Bear as checkout upsell	89% cross-sell rate	Zero-cost revenue lift
P2	Investigate Birthday Sugar Panda	6% refund rate – highest in range	Reduce returns, protect margin
P3	Build email retention programme	<10% month-1 return rate	↑ LTV per acquired customer

```
In [14]: con.close()
print("Analysis complete. Connection closed.")
```

Analysis complete. Connection closed.

```
In [ ]:
```